Laborator 4
EXCEPTII in PL/SQL;
SUBPROGRAME in PL/SQL - proceduri, functii

I. Obiective

I.1 EXCEPTII in PL/SQL

4.1. Tratarea excepţiilor predefinite ale Serverului Oracle
4.2. Tratarea excepţiilor non-predefinite Oracle Server
4.3. Tratarea excepţiilor definite de utilizator
4.3.1 Propagarea excepţiilor

I.2 SUBPROGRAME in PL/SQL

4.4. Proceduri
4.5. Subprograme

II. Consideraţii teoretice

Tratarea excepţiilor

O excepţie este un identificator PL/SQL asociat unei condiţii anormale apărute în timpul execuţiei unui bloc PL/SQL. Invocarea unei excepţii are ca efect terminarea blocului, deci ieşirea din blocul PL/SQL. Pentru evitarea unor situaţii de întrerupere anormală, excepţia poate fi captată si poate fi specificată o rutină de tratare a acesteia.

O excepţie poate fi invocata în două moduri:

a. Apare o eroare Oracle şi excepţia asociată ei este automat invocată

b. Excepţia poate fi invocată în mod explicit prin instrucţiunea RAISE în cadrul blocului.

Captarea unei excepţii

Dacă excepţia este invocată în secţiunea executabilă a unui bloc se caută în cadrul secţiunii de tratare a excepţiilor o rutină de tratare asociată. Daca PL/SQL poate trata excepţia, ea nu este propagată în blocul exterior sau în mediul apelant, caz în care se consideră că execuţia blocului s-a desfăşurat cu succes.

Propagarea unei excepţii

Dacă nu există o rutină pentru tratarea ei, excepţia este propagată în mediul apelant, caz în care execuţia blocului se termină cu eşec.

Tipuri de excepţii:

Tipul	Mod de manipulare
Excepţii pre-definite asociate erorilor care apar cel mai frecvent în blocurile PL/SQL (de exemplu *NO_DATA_ FOUND*, TOO_MANY_ROWS, INVALID_CURSOR, ZERO_DIVIDE)	Nu trebuie declarate, serverul Oracle le invocă în mod automat, dar trebuie tratate în secţiune EXCEPTION.
Excepţii non-predefine recunoscute de Oracle dar tratate de utilizator cu ajutorul codului de eroare returnat (de exemplu ORA-01400).	Trebuie declarate în secţiunea declarativă. Serverul Oracle le invocă în mod automat, dar trebuie tratate în secţiune EXCEPTION.
Excepţii definite de utilizator, asociate unor condiţii specifice de prelucrare (de exemplu cazul în care valoarea stocului unui anumit produs este zero)	Trebuie declarate în secţiunea declarativă, invocate de către utilizator şi tratate în secţiunea EXCEPTION.

Tratarea tuturor excepţiilor se realizează în secţiunea EXCEPTION a blocurilor PL/SQL astfel:

```
EXCEPTION
     WHEN exception1 [OR exception2 …] THEN
        statement1 ;
            statement2 ;
        …
     [WHEN exception3 [OR exception4 …] THEN
        statement1 ;
            statement2 ;
        …]
     [WHEN OTHERS THEN
        statement1 ;
            statement2 ;
…]
```

4.1. Tratarea excepţiilor predefinite ale Serverului Oracle

Acestea nu trebuie declarate, fiind definite de către Oracle Server si invocate implicit. Lista completă a excepţiilor predefinite Oracle poate fi consultată din PL/SQL Reference.

Câteva exemple de excepţii predefinite sunt prezentate mai jos:

Numele	Numărul erorii	Descriere
NO_DATA_FOUND	ORA-01403	O instrucţiune SELECT care ar fi trebuit sa întoarcă o singura linie nu a returnat nici o linie.
TOO_MANY_ROWS	ORA-01422	O instrucţiune SELECT care ar fi trebuit sa întoarcă o singura linie a returnat mai multe linii.
INVALID_CURSOR şi CURSOR_ALREADY_OPEN	ORA-01001	Apariţia unei operaţii ilegale asupra unui cursor (de exemplu încercarea de a deschide un cursor deja deschis).

Exemple:

Să se afişeze angajatul cu codul 10. Să se trateze eroarea apărută în cazul în care nu există nici un angajat cu acest cod.

```
DECLARE
v_nume VARCHAR2(20);
BEGIN
        SELECT name INTO v_nume
        FROM emp
        WHERE empno=10;
        dbms_output.put_line(v_nume);
        EXCEPTION
                WHEN NO_DATA_FOUND THEN
                dbms_output.put_line('Nu exista angajatul cu acest ID!');
END;
```

Să se afişeze salariul angajatului cu numele King. Să se trateze eroare apărută în cazul în care există mai mulţi angajaţi cu acelaşi nume (interogarea SQL din bloc întoarce mai multe înregistrări).

```
DECLARE
        v_sal emp.sal%type;
BEGIN
        SELECT sal SAL v_sal
        FROM empi
        WHERE name='King';
        DBMS_OUTPUT.PUT_LINE('King are salariul de: '||v_sal);
        EXCEPTION
                WHEN TOO_MANY_ROWS THEN
                DBMS_OUTPUT.PUT_LINE('Exista mai multi salariati cu
        numele King! Utilizati un cursor pentru selectie!');
END;
```

4.2. Tratarea excepţiilor non-predefinite Oracle Server

Se poate capta o eroare a Serverului Oracle ce nu are asociată o excepţie predefinită asociindu-i un nume codului de eroare returnat sau folosind clauza WHEN OTHERS. În PL/SQL, directiva EXCEPTION_INIT determină compilatorul să asocieze un nume de excepţie unui număr (cod) de eroare standard a Serverului Oracle. Aceasta permite referirea erorii standard prin nume şi scrierea unei rutine de tratare a ei.

Tratarea acestor erori se realizează în 3 paşi:

1) Declararea excepţiei: se face în zona DECLARE a blocului

NUME_EXCEPTIE EXCEPTION;

2) Asocierea codului erorii cu excepţia declarată anterior: se realizează tot în zona DECLARE prin utilizarea directivei de compilare EXCEPTION_INIT:

PRAGMA EXCEPTION_INIT(NUME_EXCEPTIE, COD_EROARE);

Unde: COD_EROARE este un cod de eroare standard Oracle;

3) Tratarea excepţiei în zona EXCEPTION a blocului:

EXCEPTION

WHEN NUME_EXCEPTIE THEN .....;

Se pot utiliza 2 atribute pentru a gestiona erorile apărute:

SQLCODE – returnează codul numeric al erorii. Poate avea următoarele valori:

- 0 – nu a apărut nici o excepţie;
- 1 – este o excepţie definită de utilizator;
- +100 – excepţia NO_DATA_FOUND;
- un număr negativ – o eroare Oracle Server;

SQLERRM – returnează mesajul asociat erorii.

Aceste atribute pot fi încărcate în variabile şi inserate într-o tabelă de erori pentru vizualizare şi verificare ulterioară.

Exemplu:

Să se insereze în tabela dept un nou departament cu ID-ul 200, fără a preciza denumirea acestuia. În acest caz va apare o eroarea cu codul ORA-01400 prin care programatorul este avertizat de încălcarea unei restricţii de integritate. Această excepţie poate fi tratată astfel:

```
DECLARE
        -- se asociază un nume codului de eroare apărut
        INSERT_EXCEPT EXCEPTION;
        PRAGMA EXCEPTION_INIT(INSERT_EXCEPT, -01400);
BEGIN
        insert into dept (deptno, dname)
        values (200, NULL);
        EXCEPTION
        --se tratează eroarea prin numele său
                WHEN insert_except THEN
                DBMS_OUTPUT.PUT_LINE('Nu ati precizat informatii
        suficiente pentru departament');
                --se afişează mesajul erorii
                DBMS_OUTPUT.PUT_LINE(SQLERRM);
END;
```

4.3. Tratarea excepţiilor definite de utilizator

În PL/SQL se pot defini excepţii ale utilizatorului. Ele trebuie declarate în secţiunea declarativă a blocului şi invocate explicit prin instrucţiunea RAISE.

Etape:

1. Se declară excepţia în secţiunea declarativă:

   nume_exceptie EXCEPTION;

2. Prin instrucţiunea RAISE se invocă în mod explicit, în cadrul secţiunii executabile:

   RAISE nume_exceptie;

3. Se tratează în rutina corespunzătoare din secţiunea de tratare a excepţiilor:

   WHEN nume_exceptie THEN.....

Exemple:

Să se invoce o eroare în cazul în care utilizatorul încearcă să execute blocul PL/SQL după ora 17.

```
DECLARE
        e_exc1 EXCEPTION;
BEGIN
        IF TO_NUMBER(TO_CHAR(SYSDATE, 'HH24'))>=17 THEN
                RAISE e_exc1;
        END IF;
        EXCEPTION
                WHEN e_exc1 THEN
                dbms_output.put_line('Este ora '||TO_CHAR(SYSDATE,
        'HH24'));
                dbms_output.put_line('Operatiune permisa doar '||' in timpul
        programului');
END;
```

Exemplu:

1. Să se şteargă salariaţii asignaţi unui cod de departament inexistent în tabelul departments. Dacă nu există nici un angajat care a îndeplinit această condiţie, să se lanseze o excepţie cu mesajul „nici un angajat nu lucreaza în departament inexistent”.

2. Să se şteargă angajaţii al căror comision reprezintă mai mult decât jumătatea diferenţei de salariu dintre şeful angajatului respectiv şi angajat. Dacă nu există nici un angajat care a îndeplinit această condiţie, să se lanseze o excepţie cu mesajul „nici un angajat cu comisionul specificat”.

Invocaţi procedura RAISE_APPLICATION_ERROR în secţiuni diferite ale blocului PL/SQL.

```
DECLARE
  e_comision EXCEPTION;
  PRAGMA EXCEPTION_INIT (e_comision, -20777);
BEGIN
  DELETE FROM emp_pnu e          -- punctul b)
  WHERE sal * comm > ((SELECT sal
                       FROM    emp_pnu
                       WHERE empno = e.manager_id) – sal)/2;
  IF SQL%NOTFOUND THEN
        RAISE e_comision;
    END IF;

  DELETE FROM emp_pnu          -- punctul a)
  WHERE deptno NOT IN ( .... ); -- completati subcererea corespunzătoare
                                --ce problema poate aparea la utilizarea
lui NOT IN ?
  IF SQL%NOTFOUND THEN
    RAISE_APPLICATION_ERROR(-20777, 'nici o stergere'); --apel in
sectiunea executabila
  END IF;

EXCEPTION
   WHEN e_comision THEN
      RAISE_APPLICATION_ERROR(-20777, ' nici un angajat cu codul
specificat '); -- apel in sectiunea EXCEPTION
END;
```

Object Type TABLE Object EMP

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
EMP	EMPNO	NUMBER	-	4	0	1	-	-	-
	ENAME	VARCHAR2	10	-	-	-	✓	-	-
	JOB	VARCHAR2	9	-	-	-	✓	-	-
	MGR	NUMBER	-	4	0	-	✓	-	-
	HIREDATE	DATE	7	-	-	-	✓	-	-
	SAL	NUMBER	-	7	2	-	✓	-	-
	COMM	NUMBER	-	7	2	-	✓	-	-
	DEPTNO	NUMBER	-	2	0	-	✓	-	-

Erorile definite de utilizator pot fi tratate la nivelul aplicaţiilor ca şi erorile Oracle Server prin atribuirea de coduri cu ajutorul funcţiei:

RAISE_APPLICATION_ERROR (NR_EROARE, MESAJ);

unde NR_EROARE poate fi un număr negativ cuprins între -20000 si -20999.

În acest caz tratarea se realizează asemănător cu erorile non-predefinite Oracle Server.

Object Type TABLE Object DEPT

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPT	DEPTNO	NUMBER	-	2	0	1	-	-	-
	DNAME	VARCHAR2	40	-	-	-	✓	-	-
	LOC	VARCHAR2	13	-	-	-	✓	-	-

Exemplu:

Creaţi tabelul mesaje_pnu având o singură coloană, numită rezultate, de tip varchar2(50). Să se scrie un bloc PL/SQL prin care să se afişeze numele departamentelor dintr-o anumită locaţie care au angajaţi.

a) Dacă rezultatul interogării returnează mai mult decât o linie, atunci să se trateze excepţia şi să se insereze în tabelul mesaje_pnu textul „mai multe departamente“.

b) Dacă rezultatul interogării nu returnează nici o linie, atunci să se trateze excepţia şi să se insereze în tabelul mesaje_pnu textul „nici un departament“.

c) Dacă rezultatul interogării este o singură linie, atunci să se insereze în tabelul mesaje_pnu numele departamentului şi locația.

d) Să se trateze orice altă eroare, inserând în tabelul mesaje_pnu textul „alte erori au apărut“.

```
DECLARE
  v_nume_dep dept_pnu.dname%TYPE;
  v_locatie       dep_pnu.loc%TYPE:='&p_locatie';
BEGIN
        SELECT   dname
        INTO       v_nume_dep
        FROM      dept_pnu
        WHERE    location = v_locatie;

        INSERT   INTO mesaje_pnu (rezultate)
        VALUES   (v_nume_dep);
        EXCEPTION
                WHEN NO_DATA_FOUND THEN
                        INSERT   INTO mesaje (rezultate)
                        VALUES   ('nici un departament');
                WHEN TOO_MANY_ROWS THEN
                        INSERT   INTO mesaje (rezultate)
                        VALUES   ('mai multe departamente');
                WHEN OTHERS THEN
                        INSERT   INTO mesaje (rezultate)
                VALUES  ('alte erori au aparut');
END;
```

4.3.1 Propagarea excepţiilor

Odată excepţia declanşată în secţiunea executabilă a unui bloc, se caută în cadrul secţiunii de tratare a excepţiilor (EXCEPTION) o rutină de tratare asociată. Daca PL/SQL poate trata excepţia, ea nu este propagată în blocul exterior sau în mediul apelant, caz în care se consideră că execuţia blocului s-a desfăşurat cu succes.

Atunci când un sub-bloc tratează o excepţie, se termină normal iar execuţia se reia în blocul ce-l cuprinde imediat după instrucţiunea END a sub-blocului.

Daca apare o excepţie iar în blocul curent nu există o rutină pentru tratarea sa, execuţia blocului se termina cu eşec, iar excepţia se propagă succesiv în blocurile exterioare până este găsită într-unul din ele o rutină pentru tratarea ei. Dacă nu se găseşte nici una, în mediul apelant apare o situaţie de excepţie nerezolvată, utilizatorul putând observa mesajul de eroare care a întrerupt execuţia normală.

Să se reţină în tabelul mesaje_pnu raportul dintre salariu şi comision pentru angajatul având cea mai mare vechime.

Varianta 1:

```
DECLARE
        v_var NUMBER(10,3);
BEGIN
        SELECT sal/comm
        INTO   v_var
        FROM   emp_pnu
        WHERE hire_date = (SELECT MIN(hire_date) FROM emp_pnu);
                                <<eticheta>>
        INSERT INTO mesaje_pnu VALUES (v_var);
        EXCEPTION
        WHEN ZERO_DIVIDE THEN v_var:=0;
        GOTO <<eticheta>>; --este posibil?
END;
```

Varianta 2:

```
DECLARE
        v_var NUMBER(10,3);
BEGIN
        IF NVL(comm, 0)=0 THEN
                GOTO eticheta; --este posibil?
        END IF;
        SELECT sal/comm
        INTO   v_var
        FROM   emp_pnu
        WHERE hire_date = (SELECT MIN(hire_date) FROM emp_pnu);
        INSERT INTO mesaje_pnu VALUES (v_var); EXCEPTION
                <<eticheta>>
                WHEN ZERO_DIVIDE THEN v_var:=0;
END;
```

Observație: Instrucţiunea GOTO nu permite:

- saltul la secţiunea de tratare a unei excepţii;
- saltul de la secţiunea de tratare a unei excepţii, în blocul curent.

Comanda GOTO permite totuşi saltul de la secţiunea de tratare a unei excepţii la un bloc care include blocul curent.

Cum se poate remedia situaţia de mai sus? Schimbaţi blocul propus (utilizând un subbloc) astfel încât să funcţioneze corespunzător.

Exemple de exceptii predefinite:

1. **NO_DATA_FOUND** - Single row SELECT returned no data.
2. **too_many_rows** - Single row SELECT returned more than one row.
3. **INVALID_CURSOR** - Illegal cursor operation was attempted.
4. **VALUE_ERROR** - Arithmetic, conversion, truncation, or constraint error occurred.
5. **INVALID_NUMBER** - Conversion of a number to a character string failed.
6. **ZERO_DIVIDE** - Attempted to divide by zero.
7. **DUP_VAL_ON_INDEX** - Attempted to insert a duplicate value into a column that has a unique index.
8. **CURSOR_ALREADY_OPEN** - Attempted to open a cursor that was previously opened.
9. **NOT_LOGGED_ON** - A database call was made without being logged into Oracle.
10. **TRANSACTION_BACKED_OUT** - Usually raised when a remote portion of a transaction is rolled back.
11. **LOGIN_DENIED** - Login to Oracle failed because of invalid username and password.
12. **PROGRAM_ERROR** - Raised if PL/SQL encounters an internal problem.
13. **STORAGE_ERROR** - Raised if PL/SQL runs out of memory or if memory is corrupted.
14. **TIMEOUT_ON_RESOURCE** - Timeout occurred while Oracle was waiting for a resource.
15. **VALUE_ERROR** - Arithmetic, conversion, truncation, or constraint error occurred.
16. **OTHERS** - This is a catchall. If the error was not trapped in the previous exception traps, the error will be trapped by this statement

4.5 Subprograme

Un subprogram este un bloc PL/SQL cu nume (spre deosebire de blocurile anonime) care poate primi parametri şi poate fi invocat dintr-un anumit mediu (de exemplu, SQL*Plus, Oracle Forms, Oracle Reports etc.)

Subprogramele sunt bazate pe structura de bloc PL/SQL. Similar, ele conţin o parte declarativă opţională, o parte executabilă obligatorie şi o parte de tratare de excepţii opţională.

Exista 2 tipuri de subprograme:

1) proceduri;
2) funcţii (trebuie să conţină cel puţin o comandă RETURN);

Subprogramele pot fi:

- **locale** (în cadrul altui bloc PL/SQL sau subprogram)
- **stocate** (create cu comanda CREATE) - odată create, procedurile şi funcţiile sunt stocate în baza de date, motiv pentru care se numesc subprograme stocate.

Sintaxa simplificată pentru crearea unei proceduri este următoarea:

```
[CREATE [OR REPLACE] ] PROCEDURE nume_procedură [
(lista_parametri) ]
   {IS | AS}
   [declaraţii locale]
BEGIN
   partea executabilă
[EXCEPTION
   partea de tratare a excepţiilor]
END [nume_procedură];
```

Lista de parametri conţine specificaţii de parametri separate prin virgulă de forma:

nume_parametru mod_parametru tip_parametru;

- mod_parametru specifică dacă parametrul este:
  - de intrare (IN) – singurul care poate avea o valoare initial;
  - de intrare / ieşire (IN OUT);
  - de ieşire (OUT)
- mod_parametru are valoarea implicită IN.

În cazul în care se modifică un obiect (vizualizare, tabel etc) de care depinde un subprogram, acesta este invalidat. Revalidarea se face fie prin recrearea subprogramului fie prin comanda:

ALTER PROCEDURE nume_proc COMPILE;

Ştergerea unei proceduri se realizează prin comenzile:

DROP PROCEDURE nume_proc;

Informaţii despre procedurile deţinute de utilizatorul curent se pot obţine interogând vizualizarea USER_OBJECTS din dicţionarul datelor.

```
SELECT OBJECT_NAME, OBJECT_TYPE, STATUS
FROM    USER_OBJECTS
WHERE  OBJECT_TYPE IN ('PROCEDURE');
```

Observaţie: STATUS – starea subprogramului (validă sau invalidă).

Codul complet al unui subprogram poate fi vizualizat folosind următoarea sintaxă:

```
SELECT     TEXT
FROM       USER_SOURCE
WHERE      NAME = 'nume_subprogram'
ORDER BY LINE;
```

Eroarea apărută la compilarea unui subprogram poate fi vizualizată folosind următoarea sintaxă:

```
SELECT     LINE, POSITION, TEXT
FROM       USER_ERRORS
WHERE      NAME = 'nume';
```

Erorile pot fi vizualizate şi prin intermediul comenzii SHOW ERRORS.

Descrierea specificaţiei unui subprogram se face prin comanda DESCRIBE.

Când este apelată o procedură PL/SQL, sistemul Oracle furnizează două metode pentru definirea parametrilor actuali:

- specificarea explicită prin nume
- specificarea prin poziţie

Exemplu: **subprog(a tip_a, b_tip_b, c tip_c, d tip_d)**
- specificare prin poziţie: **subprog(var_a,var_b,var_c,var_d);**
- specificare prin nume: **subprog(b=>var_b,c=>var_c,d=>var_d,a=>var_a);**
specificare prin nume şi poziţie: **subprog(var_a,var_b,d=>var_d,c=>var_c);**

III. Desfășurarea lucrării

Testarea secvențelor utilizând baza de date personală.

Proceduri:

Probleme rezolvate

a) Proceduri locale

1. Să se declare o procedură locală într-un bloc PL/SQL anonim prin care să se introducă în tabelul DEPT_pnu o nouă înregistrare precizând, prin intermediul parametrilor, valori pentru toate câmpurile. Invocaţi procedura în cadrul blocului. Interogaţi tabelul DEPT_pnu şi apoi anulaţi modificările (ROLLBACK).

Solutie:

```
DECLARE
 PROCEDURE add_dept
        (p_cod       dept_pnu.deptno %TYPE,
         p_nume      dept_pnu.dname %TYPE,
         p_manager dept_pnu.mgr_id %TYPE,
         p_location  dept_pnu.loc_no%TYPE)
  IS
 BEGIN
  INSERT INTO dept_pnu
  VALUES (p_cod, p_nume, p_manager, p_location);
 END;

BEGIN
 Add_dept(45, 'DB Administration' , 100, 2700);
        END;
```

```
SELECT *
FROM   dept_pnu;
ROLLBACK;
```

2. Să se declare o procedură locală care are parametrii următori:

- p_rezultat (parametru de tip OUT) de tipul coloanei ename din tabelul EMP;
- p_comision (parametru de tip IN) de tipul coloanei commdin EMP, iniţializat cu NULL;
- p_cod (parametru de tip IN) de tipul coloanei emp_id din LOYEES, iniţializat cu NULL. Dacă p_comision nu este NULL atunci în p_rezultat se va memora numele salariatului care are salariul maxim printre salariaţii având comisionul respectiv. În caz contrar, în p_rezultat se va memora numele salariatului al cărui cod are valoarea dată la apelarea procedurii.

Solutie:

```
DECLARE
v_nume emp.ename%TYPE;
PROCEDURE p2l4_pnu (p_rezultat    OUT emp.ename% TYPE,
                    p_comision   IN    emp.comm%TYPE:=NULL,
                    p_cod        IN    emp.empno %TYPE:=NULL)
 IS
 BEGIN
        IF (p_comision IS NOT NULL) THEN
                SELECT ename
INTO      p_rezultat
                FROM    emp
                WHERE comm = p_comision
                AND sal = (SELECT MAX(sal)
FROM    emp
WHERE comm= p_comision);
                DBMS_OUTPUT.PUT_LINE('Numele salariatului care are
comisionul '||p_comision|| ' este '||p_rezultat);
        ELSE
                SELECT ename
                INTO      p_rezultat
                FROM    emp
                WHERE empno = p_cod;
                DBMS_OUTPUT.PUT_LINE('numele salariatului avand codul '||p_cod|| '
este '||p_rezultat);
        END IF;
 END;
```

```
BEGIN -- partea executabilă a blocului
p2l4_pnu (v_nume,0.4); --specificare prin pozitie a parametrilor actuali
        p2l4_pnu (v_nume, p_cod=>205); -- specificare prin pozitie si nume a
parametrilor actuali
END;
```

3. Să se creeze o procedură stocată fără parametri care afişează un mesaj “Programare PL/SQL”, ziua de astăzi în formatul DD-MONTH-YYYY şi ora curentă, precum şi ziua de ieri în formatul
DD-MON-YYYY.

Solutie:

```
CREATE PROCEDURE first_pnu IS
        azi DATE := SYSDATE;
        ieri azi%TYPE;
BEGIN
        DBMS_OUTPUT.PUT_LINE('Programare PL/SQL') ;
        DBMS_OUTPUT.PUT_LINE(TO_CHAR(azi, 'dd-month-yyyy
        hh24:mi:ss')); ieri := azi -1;
        DBMS_OUTPUT.PUT_LINE(TO_CHAR(ieri, 'dd-mon-yyyy'));
END;
```

La promptul SQL apelam procedura astfel: **EXECUTE first_pnu;**

4. a) Să se creeze o procedură stocată care calculează salariul mediu al angajaţilor, returnându-l prin intermediul unui parametru de tip OUT.
b) Să se apeleze procedura regăsind valoarea medie a salariilor într-o variabilă gazdă. Afişaţi valoarea variabilei.

Soluţie:

```
CREATE OR REPLACE PROCEDURE problema4_pnu (p_salAvg OUT
emp.sal%TYPE)
AS
BEGIN
   SELECT AVG(sal)
   INTO p_salAvg
   FROM emp;
END;
```

La prompt-ul SQL:

```
VARIABLE g_medie NUMBER
EXECUTE problema4_pnu (:g_medie)
PRINT g_medie
```

5. a) Creaţi o procedură numită GET_EMP_pnu care întoarce salariul şi denumirea job-ului pentru un angajat al cărui cod este transmis ca parametru.
b) Executaţi procedura utilizând câte o variabilă gazdă pentru cei doi parametri de tip OUT.
c) testaţi procedura atât pentru coduri existente cât şi pentru coduri inexistente din tabelul EMP_pnu. Ce se întâmplă atunci când o invocăm pentru un cod inexistent?

6. a) Analizaţi subprogramele create anterior în vizualizarea din dicţionarul datelor USER_OBJECTS.
b) Regăsiţi codul unuia dintre subprogramele create anterior în vizualizarea USER_SOURCE.
c) Aflaţi tipul parametrilor uneia dintre procedurile create anterior utilizând comanda DESCRIBE.

a). Funcţii locale

1. Să se creeze o procedură stocată care pentru un anumit cod de departament (dat ca parametru) calculează prin intermediul unor funcţii locale numărul de salariaţi care lucrează în el, suma salariilor şi numărul managerilor salariaţilor care lucrează în departamentul respectiv.
Soluţie:

```
CREATE OR REPLACE PROCEDURE problema1_pnu (p_dept emp.deptno%TYPE)
AS
  FUNCTION nrSal (v_dept emp.deptno %TYPE)
     RETURN NUMBER IS
       v_numar NUMBER(3);
     BEGIN
       SELECT COUNT(*)
       INTO     v_numar
       FROM     emp
       WHERE deptno = v_dept;
       RETURN v_numar;
  END nrSal;
  FUNCTION sumaSal(v_dept emp.deptno %TYPE)
     RETURN NUMBER IS
        v_suma emp.sal%TYPE;
     BEGIN
       SELECT SUM(sal)
       INTO       v_suma
       FROM      emp
       WHERE deptno = v_dept;
        RETURN v_suma;
  END sumaSal;
  FUNCTION nrMgr(v_dept emp.deptno %TYPE)
     RETURN NUMBER IS
       v_numar NUMBER(3);
     BEGIN
       SELECT COUNT(DISTINCT mgr)
       INTO     v_numar
       FROM     emp
       WHERE deptno = v_dept;
       RETURN v_numar;
END nrMgr;
```

```
BEGIN -- partea executabila a procedurii problema1_pnu
  DBMS_OUTPUT.PUT_LINE('Numarul salariatilor care lucreaza in departamentul
'||p_dept|| ' este '|| nrSal(p_dept));
  DBMS_OUTPUT.PUT_LINE('Suma salariilor angajatilor din departamentul '|| p_dept || ' este '||
sumaSal(p_dept));
  DBMS_OUTPUT.PUT_LINE('Numarul de manageri din departamentul '|| p_dept || '
este '|| nrMgr(p_dept));
END;
```

EXECUTE **problema1_pnu**(50);

2. Să se creeze două funcţii (locale) supraîncărcate (overloaded) care să calculeze media salariilor astfel:

a) prima funcţie va avea ca argument codul departamentului, adică funcţia calculează media salariilor din departamentul specificat
b) a doua funcţie va avea două argumente, unul reprezentând codul departamentului, iar celălalt reprezentând job-ul, adică funcţia va calcula media salariilor dintr-un anumit departament şi care aparţin unui job specific

Soluție:

```
DECLARE
        medie1 NUMBER(10,2);
        medie2 NUMBER(10,2);
FUNCTION medie (v_dept emp.deptno%TYPE)
        RETURN NUMBER IS
        rezultat NUMBER(10,2);
        BEGIN
                SELECT AVG(sal)
                INTO   rezultat
                FROM   emp
                WHERE deptno = v_dept;
                RETURN rezultat;
        END;
 FUNCTION medie (v_dept emp.deptno%TYPE, v_job emp.job_id %TYPE)
        RETURN NUMBER IS
        rezultat NUMBER(10,2);
        BEGIN
                SELECT AVG(sal)
                INTO   rezultat
                FROM   emp
                WHERE deptno = v_dept AND job = v_job;
                RETURN rezultat;
        END;
```

```
BEGIN
        medie1:=medie(80);
        DBMS_OUTPUT.PUT_LINE('Media salariilor din departamentul 80 este ' ||
medie1);
        medie2 := medie(80, 'SA_REP');
        DBMS_OUTPUT.PUT_LINE('Media salariilor reprezentantilor de vanzari
                                din departamentul 80 este ' || medie2);
END;
```

Funcţii stocate

3. Să se creeze o funcţie stocată care determină numărul de salariaţi din emp angajaţi după 1995, într-un departament dat ca parametru. Să se apeleze această funcţie prin diferite modalităţi:
- printr-o comandă SELECT;
- într-un bloc PL/SQL.

Soluție:

```
CREATE OR REPLACE FUNCTION problema3_pnu (p_dept emp.deptno%TYPE)
 RETURN NUMBER
  IS
 rezultat NUMBER;
BEGIN
 SELECT COUNT(*)
 INTO   rezultat
 FROM   emp
 WHERE  deptno=p_dept
 AND    TO_CHAR(hire_date,'yyyy')>1995;
 RETURN rezultat;
END problema8_pnu;
```

```
1.  SELECT problema3_pnu (80)
    FROM dual;

2.      DECLARE
                p_dep NUMBER :=50;
                nr NUMBER;
                v_dep emp.deptno%TYPE;
    BEGIN
                nr := problema3_pnu (v_dep);
                IF nr<>0 THEN
                        DBMS_OUTPUT.PUT_LINE('numarul salariatilor angajati după
1995 in departamentul '||v_dep || ' este '||nr);
                ELSE
                        DBMS_OUTPUT.PUT_LINE('departamentul cu numarul '|| v_dep
1.      ' nu are angajati');
                END IF;
END;
```

4.Să se calculeze recursiv numărul de permutări ale unei mulţimi cu n elemente, unde n va fi transmis ca parametru.

Soluție:

```
CREATE OR REPLACE FUNCTION permutari_pnu(p_n NUMBER)
RETURN INTEGER IS
BEGIN
IF (p_n=0) THEN
                RETURN 1;
        ELSE
                RETURN p_n*permutari_pnu (p_n-1);
        END IF;
END permutari_pnu;
```

```
DECLARE g_n NUMBER;
BEGIN
        g_n := permutari_pnu (5);
        DBMS_OUTPUT.PUT_LINE('permutari ' || g_n);
END;
```

5. Să se afişeze numele, job-ul şi salariul angajaţilor al căror salariu este mai mare decât media salariilor din tabelul emp.

Soluție:

```
CREATE OR REPLACE FUNCTION medie_pnu
  RETURN NUMBER IS
medie NUMBER;
BEGIN
  SELECT AVG(sal)
  INTO      medie
  FROM   emp;
  RETURN medie;
END;
```

```
SELECT ename, job, sal
FROM   emp
WHERE sal >= medie_pnu;
```

6. a) Analizaţi subprogramele create anterior în vizualizarea din dicţionarul datelor USER_OBJECTS.

b) Regăsiţi codul unuia dintre subprogramele create anterior în vizualizarea USER_SOURCE.

c) Aflaţi tipul parametrilor uneia dintre procedurile create anterior utilizând comanda DESCRIBE.

Probleme propuse spre rezolvare

3. Să se mărească cu 40% salariul unui angajat al cărui cod este introdus de la tastatură. Să se trateze cazul în care nu există angajatul al cărui cod este specificat.

Tratarea excepţiei se va face în secţiunea executabilă.

4. Să se afişeze numele departamentului şi numele managerului unui departament al cărui cod este dat ca valoare specificata. Să se trateze cazul în care nu există departamentul al cărui cod este specificat. Tratarea excepţie se va face în secţiunea de tratare a erorilor.

5. Să se creeze un bloc PL/SQL prin care se afişează numărul de departamente care au suma salariilor mai mare decât o valoare dată. Să se trateze cazul în care nici un departament nu îndeplineşte această condiţie.

6. Să se creeze un bloc PL/SQL prin care să se afişeze numele departamentului care funcţionează într-o anumită locaţie. Dacă rezultatul interogării nu returnează nici o linie, atunci să se trateze excepţia şi să se insereze în tabelul erori_pnu codul erorii -20001 cu mesajul “nu exista departamente in locatia data”. Dacă rezultatul interogării este o singură linie, atunci să se afişeze numele departamentului. Dacă rezultatul interogării presupune mai multe linii, atunci să se introducă în tabelul erori_pnu codul erorii -20002 cu mesajul “exista mai multe departamente in locatia data”.

Proceduri:

1. Să se creeze o procedură stocată care măreşte salariile angajaţilor care nu au comision şi au media mai mică decât cea a departamentului în care lucrează cu o valoare transmisă ca parametru.

2. Să se declare o procedură locală care are următorii parametri:

- p_rezultat (parametru de ieşire) de tip NUMBER;

- p_job (parametru de intrare) de tip job din jobs_pnu, iniţializat cu NULL;
- p_titlu (parametru de intrare) de tip job_title din jobs_pnu, iniţializat cu NULL.

Dacă job nu este NULL atunci în rezultat se va memora numărul de angajaţi având codul job-ului specificat ca parametru. În caz contrar, în rezultat se va memora numărul de angajaţi care au titlul job-ului dat de al treilea parametru din procedura.

Functii:

1. Să se creeze o funcţie stocată care determină numărul de salariaţi care au fost angajaţi ulterior tuturor subalternilor unui manager al cărui cod este dat ca parametru. Să se apeleze această funcţie într-un bloc PL/SQL.

2. Să se creeze trei funcţii locale cu acelaşi nume care să calculeze numărul de salariaţi astfel:
- prima funcţie va avea ca argument codul departamentului, adică funcţia calculează numărul de salariaţi din departamentul specificat;
- a doua funcţie va avea două argumente, unul reprezentând codul departamentului, iar celălalt reprezentând anul angajării, adică funcţia va calcula numărul de salariaţi din departament şi care au fost angajaţi într-un anumit an;
- a treia funcţie va avea trei argumente, unul reprezentând codul departamentului, unul reprezentând anul angajării, iar celălalt grila de salarizare, adică funcţia va calcula numărul de salariaţi din departament, care au fost angajaţi într-un anumit an şi au salariul într-o anumită grilă de salarizare.

3.Să se creeze o funcţie pentru calculul recursiv al combinărilor.

4. Să se modifice salariul unui angajat al cărui cod este introdus ca parametru astfel încât să devină media salariilor angajaţilor care câştigă comision dintr-un departament dat ca parametru.

5. Să se scrie o funcţie care să întoarcă, pentru un angajat al cărui cod este specificat ca parametru, vechimea exprimată în luni. Să se utilizeze funcţia într-o instrucţiune SELECT care să întoarcă numele angajaţilor, salariul şi numărul de luni lucrate.